Royaume du Maroc
Université Sultan Moulay Slimane
École Nationale des Sciences Appliquées
Béni Mellal

Rapport de Travaux Pratiques

TP 01 – POO en Java

Module : Génie Logiciel

Filière : « Transformation digitale industrielle **»**

Niveau : 2ème Cycle d'Ingénieur

Année Universitaire : 2025-2026

Réalisé par :

CHAHBI Salma

Demandé par :

Pr. Bahaa Eddine ELBAGHAZAOUI

Exercice 1.1 – Gestion d'animaux dans un zoo

✓ Partie 1 : Définitions théoriques

Q1 – Encapsulation

L'encapsulation consiste à regrouper données et méthodes dans une classe tout en restreignant l'accès direct aux données. Dans Animal, l'attribut nom est private : il n'est accessible que via getNom(). Cela protège l'intégrité de la donnée.

Q2 – Héritage

L'héritage permet à une sous-classe de réutiliser les attributs et méthodes d'une super-classe. Ici Mammifere et Oiseau héritent de Animal via le mot-clé extends. Elles héritent de l'attribut nom et de la méthode getNom().

Q3 – Abstraction

L'abstraction consiste à définir un modèle général en masquant les détails d'implémentation. Animal est déclarée abstract : elle ne peut pas être instanciée. La méthode faireDuBruit() est abstraite, forçant chaque sous-classe à fournir sa propre implémentation.

Q4 – Polymorphisme

Le polymorphisme permet d'utiliser une référence de type parent pour manipuler des objets de types différents. Dans main(), tigre et perroquet sont déclarés Animal mais appellent chacun la version redéfinie de faireDuBruit() ("Tigre grogne." / "Perroquet chante.").

✓ Partie 2 : Code complété

Q5 – Méthode voler() + Q6 – Appel dans main()

```
public void voler() {
System.out.println("L'oiseau vole.");
}
```

```
// Caster la référence Animal en Oiseau pour accéder à voler()
Oiseau perroquetOiseau = (Oiseau) perroquet;
perroquetOiseau.voler(); // Sortie : L'oiseau vole.
```

✓ Partie 3 : Identification des mots-clés

Concept	Mot-clé Java	Exemple dans le code
Héritage	extends	class Mammifere extends Animal
Redéfinition de méthode	@Override	@Override public void faireDuBruit()
Abstraction	abstract	abstract class Animal

Exercice 1.2 - Programmation Orienté Objet (Code java)

✓ Partie 1 : Théorie et concepts

Q1 – Encapsulation

Les attributs marque, modele, annee de Vehicule sont private et accessibles uniquement via les getters publics. nombreDePortes (Voiture) et capaciteDeCharge (Camion) sont également privés.

Q2 – Héritage

Voiture et Camion héritent de Vehicule via extends. Le constructeur parent est appelé avec super(...) pour initialiser marque, modele et annee.

Q3 – Polymorphisme

maVoiture et monCamion sont déclarés de type Vehicule mais appellent chacun la version redéfinie de afficherDetails() grâce à la liaison dynamique (late binding).

Q4 – Abstraction

Vehicule est abstraite car il n'existe pas de véhicule générique sans type précis. La méthode afficherDetails() est abstraite : chaque sous-classe doit définir comment elle affiche ses propres détails.

✓ Partie 2 : Code complété

Q5 – Méthode demarrer() dans Vehicule

```
public void demarrer() {
System.out.println("Le véhicule démarre.");
}
```

Q6 – Nouvelle classe Moto

```
class Moto extends Vehicule {
private String typeDeGuidon;
public Moto(String marque, String modele, int annee, String typeDeGuidon) {
super(marque, modele, annee);
this.typeDeGuidon = typeDeGuidon;
}
@Override
public void afficherDetails() {
demarrer();

System.out.println("Moto: " + getMarque() + " " + getModele()
+ " (" + getAnnee() + "), Guidon: " + typeDeGuidon);
}
}
```

- ✓ **Partie 3 : Identification des mots-clés**

Concept	Mot-clé Java	Exemple dans le code
Redéfinition de méthode	@Override	`@Override public void afficherDetails()`
Héritage	extends	`class Voiture extends Vehicule`
Appel constructeur parent	super(...)	`super(marque, modele, annee)`

Exercice 1.3 - Gestion de comptes bancaires

- **Description de l'implémentation :**

Ce programme modélise un système bancaire avec deux types de comptes. La classe CompteBancaire sert de classe de base avec les opérations fondamentales (déposer, retirer). La classe CompteEpargne en hérite et ajoute le calcul d'intérêts. La classe CompteCourant redéfinit le retrait pour autoriser un découvert.

- **Concepts POO appliqués**

Encapsulation

Les attributs numeroCompte et solde sont private. Le solde ne peut être modifié que via deposer() et retirer(), qui incluent des validations (montant positif, solde suffisant).

Héritage

CompteEpargne et CompteCourant héritent de CompteBancaire et réutilisent ses méthodes de base. CompteCourant redéfinit retirer() pour gérer le découvert autorisé.

Polymorphisme

Les deux types de comptes peuvent être manipulés via des références CompteBancaire. La méthode retirer() se comporte différemment selon le type réel de l'objet (vérification simple du solde pour CompteEpargne, découvert toléré pour CompteCourant).

- **Exemple de sortie attendue**

```
========== COMPTE COURANT ==========
Depot de 150.0 euros. Solde: 650.0
Retrait de 100.0 euros. Solde: 550.0

========== COMPTE EPARGNE ==========
Depot de 500.0 euros. Solde: 1500.0
Retrait de 200.0 euros. Solde: 1300.0
Interets a 3.5% : +45.5 euros
```